

Analysis: Imbalance Obviation

Test Set 1

For Test Set 1, we can enumerate all 2^N possible strings of length N composed only of L s and/or R s, and check whether each one would break the scale, by simulating the process explained in the problem statement. We can then output any one string that works; the statement guarantees that at least one exists.

This process has a time complexity of $O(N \times 2^N)$ for each test case, which is fast enough for this test set.

Test Set 2

Suppose we've placed our first marble on one of the pans. Then the second marble must go on the other pan. Now we are effectively back in the same situation — the scale is balanced and we have our choice of pan again, but then that choice forces the next choice, and so on.

Similarly, if the number of marbles is even (as is always the case in this test set), the scale is balanced before we start to remove marbles. This means we can remove the first marble from either pan, but the second marble we remove must come from the other one. The third marble can be removed from either one again, but the fourth must be removed from the other one, and so on.

We can formalize the first observation by saying that marbles $2i$ and $2i-1$ must go on different pans, for every $i \geq 1$. We can formalize the second observation by saying that marbles A_{2i-1} and A_{2i} must go on different pans, for every $i \geq 1$.

The rules above help to reduce this problem to [graph coloring](#). In our graph, each node represents a marble, and each pair of marbles that are required to be on different pans is connected by an edge. The resulting graph has N nodes and at most N edges, and each of the nodes is connected to at most 2 other nodes. If we can 2-color the graph, we can use one of the colors (it doesn't matter which one) to represent the left pan and the other to represent the right pan, and then we will have our desired assignment of marbles to pans. Better yet, 2-coloring takes at most linear time!

How can be sure that a 2-coloring is possible, apart from the assurances in the problem statement that a solution always exists? Well, this is possible if and only if there are no cycles of odd length — can we show that?

The constraints from the marble-adding phase pair up nodes 1 and 2, 3 and 4, etc. Call these the "original pairs". Any additional constraint from the marble-removing phase (that is, from the input permutation A) can do one of the following: match an original pair's edge, in which case that pair cannot have other connections to the rest of the graph and is not a cycle, or add an edge that connects two of the original pairs. Then, any path that does not repeat edges alternates between edges connecting original pairs and edges from the marble-removing phase that do not connect original pairs. Therefore, any cycle formed is of even length, or else it would have two edges of the same type one after the other.

So, the resulting graph is [bipartite](#) and our strategy will always find a solution.

Test Set 3

Let's reconsider the above strategy and see if we can make it work for odd N :

- The first rule ("marbles $2i$ and $2i-1$ must go on different pans, for every $i \geq 1$ ") is still fine — the pans will be balanced right before we place marble N , so it does not matter where we put that marble.
- After we remove the marble A_1 (more on that in a moment), the pans will be balanced again, so we need our second and third marbles to be removed from opposite sides, and then our fourth and fifth, and so on. We can replace the second rule above ("marbles A_{2i-1} and A_{2i} must go on different pans, for every $i \geq 1$ ") with: marbles A_{2i} and A_{2i+1} must go on different pans, for every $i \geq 1$. (If we write this as "marbles A_{N-2i+2} and A_{N-2i+1} must go on different pans, for every $i \geq 1$ ", then the condition is the same as in the even- N case.)
- Finally, the first marble removed, A_1 , must come from the heaviest pan. We know that marble N is in the heaviest pan, so one way to represent this constraint is to replace both A_1 and N in the graph with a new "node" that represents them both. We can do this because we know that marble N has no constraints arising from the marble-adding phase, and marble A_1 has no constraints arising from the marble-removing phase. So the argument from earlier about no odd cycles still holds; this new "node" can only be entered via one type of constraint and exited via another.

A less complex version of the above idea is: we can introduce an imaginary extra marble that we add last and remove first, making the total number of marbles even again. We can add $N + 1$ to the end of the identity permutation, and to the beginning of the input permutation (since it was the last marble added, it's always safe to remove it first, restoring a previous valid state). Then we can solve the problem as in the even version above, and finally chop off the last character of the sequence. This gives us the same $O(N)$ time complexity as in Test Set 2.

A note on 2-SAT

A more abstract view of the same solution is to represent each marble as a Boolean variable (where true corresponds to assignment to one pan and false to assignment to the other pan), and then encode the requirements explained above as two implications of literals, which are in turn disjunctions of literals. The conjunction of all those requirements is a valid input to [2-satisfiability](#) (2-SAT). Notice that the graph created in the typical 2-SAT algorithm is exactly 2 copies of the graph mentioned for the solution above, so the algorithm ends up being the same in both cases.

Analysis: Impromptu Outdoor Gallery

We can start by making the observation that for every valid quadrilateral, $p_1p_2p_3p_4$, at least one of the two diagonals divides the quadrilateral into two triangles. We can order the points in the quadrilateral such that the diagonal formed by points p_1 and p_3 splits the quadrilateral into two triangles. This means that points p_2 and p_4 are on opposite sides of the infinite line through p_1 and p_3 or vice versa. Without loss of generality, we can assume the former, since it is just a renaming of the same set of points in the same order. The area of $p_1p_2p_3p_4$ is equal to the sum of the areas of the two triangles $p_1p_3p_2$ and $p_1p_3p_4$. These areas are equal to half of the product of the distance between p_1 and p_3 and the distance between p_2 (or p_4) and the line through p_1 and p_3 . Therefore, if we choose two points to be p_1 and p_3 in our quadrilateral, we can find the minimum area using those two points by getting the closest point to the line through p_1 and p_3 on each side of the line.

Test set 1

With only 25 points, we can test every set of 4 points in every order. To make sure a specific order defines a simple polygon, it suffices to check the sign of the signed area of both triangles is positive. This can be done by checking the sign of the [cross product](#) when computing the area of the two triangles that make up the quadrilateral.

We could also loop over all possible pairs of points to use as p_1 and p_3 , and then, for each pair, loop through the remaining points to find the closest on each side. There are $O(N^2)$ unique pairs of points and for each pair, we check $O(N)$ other points to find the closest on each side. This gives an overall time complexity of $O(N^3)$.

Test set 2

In test set 2 we have significantly more points to choose from that a time complexity of $O(N^3)$ will not work for the given time limit. We can, however, precompute all lines formed by unique pairs of points and do a circular [sweep line](#), processing those lines in order of their slopes; this will make it much faster to find the closest points to the lines. As we rotate, we keep the points in a list sorted by their distance to the line.

We can add the horizontal line to the set of stop points, and use it as the starting angle for the sweep line. This simplifies initialization because sorting the points by distance to this line is simply sorting by y-coordinate. Then, for any p_1 and p_3 , the closest points to the line p_1p_3 must be ones which are adjacent to either p_1 or p_3 in the sorted list (because p_1 and p_3 have a distance of 0). There is a constant number of candidates, so we can check them all in constant time to find the best p_2 and p_4 .

As we rotate our orientation to line up with the next smallest sloped line, we can see that the only points that change their ordering in the sorted list are the two points of the line segment being looked at. Taking advantage of this, we can maintain our sorted list of points by just swapping the adjacent pairs of each line after we consider them in order of slope.

Because we still have $O(N^2)$ unique lines and we need to sort them by their slopes, the initial time complexity to generate the sorted list of lines and the initial ordering of the points is $O(N^2 \log N)$. After we have all the line segments sorted, we can maintain the sorted list and go

through each line in order and find the closest points in $O(1)$ per line. This results in an overall time complexity of $O(N^2 \log N)$.

Analysis: Interleaved Output: Part 1

Test Set 1

One way of approaching this test set is to try all ways of assigning pairs of characters in the string (an uppercase I or lowercase i followed by an uppercase O or lowercase o) to particular computers, and in each case, recursively check whether the remaining string could have been produced. If, at any point during our check, our string starts with an uppercase O or lowercase o , then the string is invalid and we can abandon that branch of the search.

For example, if we have $IoiO$, we can try assigning the uppercase I and the uppercase O to an IO computer, then recursively check the remaining oi and find that it's invalid, then try instead assigning the uppercase I and lowercase o to an Io computer, and so on. Finally, find the largest possible number of advertisements of IO , across all valid assignments.

Test Set 2

The above strategy is too slow for Test Set 2 because there are too many possible pairs to check. Let's look for a simpler strategy.

As we scan through a string, whenever we encounter an uppercase or lowercase O , we must find a previous unused uppercase or lowercase I to pair it with (implicitly claiming that a particular computer printed both of those characters). Intuitively, to find an interpretation that maximizes the number of times IO is advertised, we would like to do the following as much as possible:

- Preferentially pair an uppercase O with an uppercase I , rather than with a lowercase i
- Preferentially pair a lowercase o with a lowercase i , rather than with an uppercase I

However, we may not always be able to get what we want! For example, suppose we are scanning through the input $IoiO$. When we reach the lowercase o , we have no choice but to pair it with the preceding I .

We can prove, though, that if we adhere to the above preferences whenever we have a choice, we will find the correct answer. Suppose we are carrying out this method, and we reach an uppercase O — call it O_1 — that we can match to either some previous unmatched uppercase I — call it I_{prev} — or some previous unmatched lowercase i — call it i_{prev} . Suppose we choose to match O_1 with i_{prev} . Then we must eventually match I_{prev} with some later uppercase or lowercase O . (It is guaranteed that at least one of these exists because the input satisfies a balanced parentheses constraint, as outlined in the Limits section.) Call that later O O_2 . We will obtain 1 "point" (i.e. we will be able to claim that an IO computer advertised its event once) if O_2 is uppercase, and 0 points if O_2 is lowercase.

But then observe that we could have instead matched the uppercase O_1 with I_{prev} (scoring 1 point *for sure*), and matched the lowercase i_{prev} with the same O_2 . This argument holds true no matter which O_2 we picked. So preferentially matching with the I_{prev} is no worse, and may be better.

A similar argument holds for the situation in which we reach a lowercase o that we can match to either a previous uppercase I or a previous lowercase i , and it tells us to preferentially match

with the i_{prev} .

We have covered the only two cases in which we can make a decision. Since no other strategy is better in either case, our strategy is an optimal one. It is possible that we may have a choice of e.g. two previous uppercase $\mathbb{I}$ s and one previous lowercase $\mathbb{i}$, but then it does not matter which of the uppercase $\mathbb{I}$ s we pick, since whichever one we don't use will still be "previous" for the purposes of later decisions.

A simpler way to frame this strategy is as follows: scan through the string from left to right, keeping two counts: $C_{\mathbb{I}}$, the number of unmatched uppercase $\mathbb{I}$ s seen so far, and $C_{\mathbb{i}}$, the number of unmatched lowercase $\mathbb{i}$ s seen so far. When we encounter an uppercase $\circ$, score one point and decrement $C_{\mathbb{I}}$ if $C_{\mathbb{I}}$ is positive, or otherwise decrement $C_{\mathbb{i}}$. When we encounter a lowercase $\circ$, decrement $C_{\mathbb{i}}$ if $C_{\mathbb{i}}$ is positive, or otherwise decrement $C_{\mathbb{I}}$. Notice that the rules of the problem guarantee that these counts will never simultaneously become zero at the time we encounter an uppercase $\circ$ or lowercase $\circ$.

Analysis: Interleaved Output: Part 2

An important difference between this problem and the Part 1 version is that in this version, for each of the events, there is only a single computer that can print it. That is, there are only four computers printing simultaneously for the four events, IO , io , $I\circ$ and $i\circ$. Our optimal greedy strategy from the Part 1 analysis is not compatible with this constraint.

For example, consider the sample input $Ii\circ ioI\circ O$. Our Part 1 strategy would find an interpretation in which $I\circ$ was advertised twice. But this interpretation requires two separate computers to have printed io , since one computer cannot print an i followed by another i ... and in Part 2, we only have one io computer.

Let us find a way to turn this new constraint into an advantage. Regardless of how many times each computer ends up advertising its event, each computer can be in only one of two states at any given time: either it is about to print its first character, or about to print its second character. So, at any point, the system of 4 computers is in one of $2^4 = 16$ states. This situation, in which we have only a limited number of states to consider, is ideal for a [dynamic programming](#) approach in which we step through the string and keep track of the best option seen so far for each of those states.

Let us describe our 16 states in terms of which computers are about to print their *second* characters. For example, the state $\{IO, io\}$ means that the IO and io computers need to print their second character ($\circ$) next, and the $i\circ$ and $I\circ$ computers need to print their first character (I or i) next.

We will also define the "score" of a prefix of the input string p and a state s as the number of completed IO events designated after processing p ending in state s . Since scores only increase, we only care about the maximum score for each combination. The final answer to the problem is the score for $p = S$ and $s = \{\}$, that is, when we have processed the entire string and no computer has printing pending. We will also define the "score" of a partial string as the number of completed IO events designated so far in that string.

Consider the input string $Ii\circ ioI\circ O$. When we process the first I , we must decide which computer printed that character — that is, after that choice, we can either be in the state $\{IO\}$ or the state $\{I\circ\}$. With dynamic programming, we simultaneously keep track of all of these possible worlds; we are not actually committing to a particular choice. After we process the next character, depending on our interpretation, we can be in one of four states: $\{IO, io\}$, $\{IO, i\circ\}$, $\{I\circ, io\}$, or $\{I\circ, i\circ\}$. Then, moving on to the next $\circ$, we find that $\{I\circ, io\}$ is inconsistent, and the possible states become $\{IO\}$ (with a score of 0 so far), $\{io\}$ (with a score of 1), $\{i\circ\}$ (with a score of 0), or $\{I\circ\}$ (with a score of 0). We continue this until we have finished processing the string, and then we have our answer.

Our dynamic programming approach has $O(L)$ stages (where L is the input string length), and each stage has a constant processing time since there are no more than 16 states to consider. This results in an overall time complexity of $O(L)$, but notice that we have only been able to wave away a constant factor because the number of events/computers was small. With E possible events, the complexity would be $O(L \times 2^E)$.